acceptable ones like 404. However, in that case, you don't want to retry the request. Project Reactor provides an `onErrorResume()` operator to define a fallback when a specific error occurs. You can add it to the reactive stream after the `timeout()` operator and before the `retryWhen()` so that if a 404 response is received (`WebClientResponseException.NotFound` exception), the retry operator is not triggered. Then you can use the same operator again at the end of the stream to catch any other exception and fall back to an empty `Mono`. Update the `getBookByIsbn()` method in the `BookClient` class as follows.

Listing 8.26 Defining exception handling and fallbacks for the HTTP call

```
public Mono<Book> getBookByIsbn(String isbn) {
  return webClient
    .get()
    .uri(BOOKS_ROOT_API + isbn)
    .retrieve()
    .bodyToMono(Book.class)
    .timeout(Duration.ofSeconds(3), Mono.empty())
    .onErrorResume(WebClientResponseException.NotFound.class,
      exception -> Mono.empty())                                    ❶
    .retryWhen(Retry.backoff(3, Duration.ofMillis(100)))
    .onErrorResume(Exception.class,
      exception -> Mono.empty());                                   ❷
}
```

❶ Returns an empty object when a 404 response is received

❷ If any error happens after the 3 retry attempts, catch the exception and return an empty object.

NOTE In a real-world scenario, you would probably want to return some contextual information depending on the type of error, instead of always returning an empty object. For example, you could add a `reason` field to the `Order` object to describe why it's been rejected. Was it because the book is unavailable in the catalog or because of network problems? In the second case, you could inform the user that the order cannot be processed because it's momentarily unable to check the book's availability. A better option would be to save the order in a pending state, queue the order submission request, and try it again later, using one of the strategies I'll cover in chapter 10.